LAB6: GridSearch and Ensemble Learning

This practical assignment consists of two parts:

1. Part I
    - Grid search
    - Ensemble Learning
    - Ensemble Learning and Grid Search
2. part II
    - Project: 1 hour 30 minutes

I-Decision Tree hyperparameters

Exercise 1 :

Find the best hyperparameters for a decision tree applied to the Breast Cancer dataset. But be careful: several methodological and technical traps have been deliberately hidden. Your goal is to identify them, correct them, and understand why.

Objective

based on the Cancer dataset, You want to build a powerful model to predict whether a tumour is benign or malignant. You decide to use a *DecisionTreeClassifier* and a *GridSearchCV*.

Instructions

- Write your code based on these steps

    Step 1: a) load from sklean.datasets the Breast cancer dataset after importing load_breast_cancer data object and explore the data
    - Display the number of observations, variables, and classes.

    Step 2: Split the data into train and test subsets using *train_test_split()* function with:

```
    X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y)

```

Step 3: Create the base model

- **DecisionTreeClassifier(random_state=42)** without optimization

Step 4: Define the hyperparametr's Grid to be testes

- Example

```
     param_grid = {
        'criterion': ['gini', 'entropy', 'log_loss'],
    'max_depth': [2, 4, 6, 8, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

```

Step 5: Lauch the Grid Search by using GridSearchCV with:

- CV=2
- scoring='accuracy'
- refit=True
- Verbose=1

Step 6: Print the best parameters and the mean Cross Validation (CV) score using the Constants **best_param_** and **best_score_** on your object grid created before;

step 7: Evaluate on the test dataset

Step 8: Visualize the optimal Tree using **sklearn.tree.plot_tree**

- Detect and correct at minimum 7 traps in your code!
- What do you notice when you increase max_depth and decrease min_samples_leaf? For example take max_depth =None and min_samples_leaf=1?
- What happens if min_samples_split and min_samples_leaf are too small? For example you can take min_samples_spli and min_samples_leaf=1!
- What we can conclude about the trade-off between Bias and Variance?

Exercise 2: SVM and Grid Search

Implement the same pipeline using SVM instead of a Decision Tree and Compare the optimal Decision Tree to the optimal SVM on the basis of the F1-score, the inference time and the decision interpretability.

Instructions

1. uses the breast cancer dataset (breast_cancer from scikit-learn).
2. performs hyperparameter search (GridSearchCV) for SVM and Decision Tree.
3. compares the best models on an independent test set.
4. displays performance and confusion matrices.

Exercise 3: Ensemble Learning — Bagging and Voting with SVM and Decision Tree

Objective

The goal of this exercise is to implement an **ensemble learning** approach for a real-world classification problem. Students will use the `Breast Cancer Dataset` from `scikit-learn` and compare several ensemble strategies:

- **Bagging** applied to a **Support Vector Machine (SVM)**
- **Bagging** applied to a **Decision Tree**
- A **hybrid ensemble** combining both using a **VotingClassifier**

Dataset

We use the built-in dataset from `scikit-learn`:

```
from sklearn.datasets import load_breast_cancer
```

This dataset contains features extracted from digitized images of breast tumor cells. The target variable indicates whether the tumor is `malignant (1)` or `benign (0)`.

Tasks

1. **Data loading and preprocessing**
   - Load the dataset and split it into training and test sets (80% / 20%).
   - Standardize the features using `StandardScaler` (mandatory for SVM).
2. **Build base models**
   - Create a `SVC()` and a `DecisionTreeClassifier()`.
   - Wrap each base model inside a `BaggingClassifier()`.
3. **Hyperparameter optimization with GridSearchCV**

- Use `GridSearchCV` to find the best hyperparameters for each ensemble:
  - For SVM: $C \in \{0.1, 1, 10\}$, `kernel` $\in \{\text{linear}, \text{rbf}\}$, `gamma` $\in \{\text{scale}, \text{auto}\}$.
  - For Decision Tree: `max_depth` $\in \{3, 5, 10, None\}$, `min_samples_split` $\in \{2, 5, 10\}$.
- Also optimize Bagging parameters such as `n_estimators` and `max_samples`.
- Compare the best results in terms of Accuracy, Precision, Recall, and F1-score.

4. **Build a hybrid ensemble (VotingClassifier)**
   - Combine the two optimized models (Bagging SVM and Bagging Decision Tree) in a `VotingClassifier`.
   - Use a **soft voting** strategy (average of predicted probabilities).
   - Evaluate the overall performance of the hybrid model.
5. **Evaluation and visualization**
   - Compute and display the confusion matrices for all models.
   - Compare the test accuracies in a bar chart.
   - Discuss:
     (a) Which model achieves the best trade-off between performance and interpretability?
     (b) Present a table comparing accuracy, inference time, model size and interpretability

Evaluation criteria

- Code readability and structure (comments, clarity, modularity)
- Correctness and consistency of results
- Ability to interpret and explain the performance differences

Expected results

- The Bagging SVM should reach an accuracy around 98–99%.
- The Bagging Decision Tree should perform around 96–98%.
- The hybrid VotingClassifier is expected to slightly improve the overall accuracy.

Possible extensions

- Visualize class boundaries in 2D using PCA reduction.
- Implement a `StackingClassifier` combining both ensembles instead of simple voting.

Part II: Apply these techniques to your project!

In this step of your project, you will

- apply grid search to establish the optimal configuration of your baseline models.
- include this step in your final proposal.
- apply ensemble models: Voting Bagging or Stacking
- include discussion of the different evaluations in your final report. So take a time to finalize your figures and your visualizations.

Do not forget to push your code into your GitHub.